

Instructions: Please answer the following questions, making sure to write legibly on the following pages. The code below is a subsection of a Python file that runs Flask for a movie theater system.

- Since we are writing Python code, please be careful about spacing/indentation. Leave enough room that it is obvious when you want things indented.
- Make sure to write legibly.
- All questions are worth equal points unless otherwise stated.
- The quiz is cumulative and may include material from previous weeks.

```
conn = get_db_connection() # returns an SQL Connection

@app.route('/api/movies/sales/<movie_title>', methods=['GET'])
def movie_sales(movie_title):
    cursor = conn.cursor()
    cursor.execute(
        f"SELECT date, tickets_sold FROM sales WHERE movie = {movie_title}")
    results = cursor.fetchall()
    # Convert tickets to thousands
    formatted_results = [(date, tickets / 1000) for date, tickets in results]
    return jsonify({'movie': movie_title, 'sales': formatted_results}), 200

@app.route('/api/movies/revenue/<movie_title>', methods=['GET'])
def movie_revenue(movie_title):
    cursor = conn.cursor()
    cursor.execute(
        f"SELECT date, revenue FROM revenue WHERE movie = {movie_title}")
    results = cursor.fetchall()
    # Convert revenue to thousands of dollars
    formatted_results = [(date, rev / 1000) for date, rev in results]
    return jsonify({'movie': movie_title, 'revenue': formatted_results}), 200

@app.route('/api/movies/concessions/<movie_title>', methods=['GET'])
def movie_concessions(movie_title):
    cursor = conn.cursor()
    cursor.execute(
        f"SELECT date, concession_revenue FROM concessions WHERE movie = {movie_title}")
    results = cursor.fetchall()
    # Convert concessions to thousands of dollars
    formatted_results = [(date, revenue / 1000) for date, revenue in results]
    return jsonify({'movie': movie_title, 'concessions': formatted_results}), 200
```

1. The function `get_top_movies()` queries the database and returns the top 3 movies by ticket sales in the following format:

```
[('Dune 2', 1234, 'PG-13'),  
 ('Barbie', 1089, 'PG-13'),  
 ('Oppenheimer', 956, 'R')]
```

Note: Each tuple contains (movie_name, tickets_sold, rating). We want to create a Flask route `/api/movies/top_movies` to return this data in the format below. Please write the complete code for this route (including the decorator). You can assume that the list returned from `get_top_movies()` is in order from highest to lowest and that the `get_top_movies()` function has already been imported.

```
{"rankings": {1: "Dune 2", 2: "Barbie", 3: "Oppenheimer"},  
 "total_tickets": 3279 }
```

2. A movie theater applies promotional discounts to ticket prices. Write a function called `apply_discounts` that calculates a final price after applying multiple discounts sequentially. The function should:

- Take a base price as the first argument
- Accept *any number* of discount percentages as additional arguments
- Apply each discount percentage in order (e.g., 10% off, then 5% off the new price)
- Return the final price

For example, `apply_discounts(100, 10)` should return 90, and `apply_discounts(100, 10, 5)` should work as follows:

- Start with 100
- Apply 10% discount: $100 - (100 * 0.10) = 90$
- Apply 5% discount: $90 - (90 * 0.05) = 85.50$
- Return 85.50

3. Write a function called `get_popular_showtimes` that takes a list of tuples as input. Each tuple contains (`movie`, `showtime`, `tickets_sold`, `theater_number`). For example:

```
[('Dune 2', '14:00', 45, 3),  
 ('Barbie', '15:30', 12, 1),  
 ('Dune 2', '18:00', 50, 5),  
 ('Oppenheimer', '20:00', 8, 2)]
```

Your function should return a list of dictionaries containing only the title and showtime where 20 or more tickets were sold. Each dictionary should have keys `'movie'` and `'time'`.

For the example above, the function should return:

```
[{'movie': 'Dune 2', 'time': '14:00'}, {'movie': 'Dune 2', 'time': '18:00'}]
```

4. The three routes shown on the first page (`movie_sales`, `movie_revenue`, and `movie_concessions`) violate the DRY principle. Note that all three routes query the database, format results, and return JSON responses with status code 200. The `get_db_connection()` function (also on page 1) is working correctly and does not need to be changed.

Please refactor this code to avoid the DRY violation. Then rewrite `movie_sales` to use your helper function. Do not rewrite all 3 functions.

Your answer should include:

- The helper function
- The rewritten `movie_sales` route